

Universidade Estadual de Londrina

Estudo - Docker

Marco Antônio Cottorello Henry

Danilo Kotaka Marana

Londrina

7 de abril de 2026

Sumário

1	Introdução	3
2	Plataforma Docker	4
2.1	Containers	4
2.2	Ciclo de desenvolvimento	4
2.3	Eficiência de recursos	4
3	O que é o Docker? Containers vs Máquinas Virtuais	5
3.1	Características dos Containers	5
3.2	Containers vs Máquinas Virtuais	5
4	Arquitetura Docker	5
4.1	Docker Daemon	6
4.2	Docker Client	6
4.3	Docker Desktop	6
4.4	Docker Registries	6
4.5	Docker Objects	6
4.5.1	Images	6
4.5.2	Containers	7
4.6	Exemplo: comando <code>docker run</code>	7
5	Instalação	8
5.1	Requisitos do sistema	8
5.2	Instalação no Ubuntu	8
6	Rodando o primeiro container e comandos de sobrevivência	8
6.1	Comandos básicos	8
6.1.1	Executando containers	8
6.1.2	Listando containers	9
6.1.3	Parando containers	9
7	Imagens Docker	9
7.1	Características das Imagens	9
7.2	Encontrando Imagens	9
7.3	Comandos básicos para imagens	10
8	Docker Registry	10
8.1	Outros registros	10
8.2	Registry vs Repository	10
8.3	Fluxo prático: build, tag e push para o registro	10
9	Entendendo as camadas de uma imagem	11
9.1	Exemplo de camadas	11
9.2	Empilhamento de camadas	11

9.3	Criando layers manualmente	11
9.3.1	Criando uma base image	11
9.3.2	Construindo uma app image	12
10	Anatomia do Dockerfile	13
10.1	Exemplo de Dockerfile	13
10.2	Principais instruções	13
10.3	Sequência típica de um Dockerfile	13
10.4	Exemplo prático: aplicação Node.js	14
11	Construindo, tagueando e publicando imagens	14
11.1	Building images	14
11.2	Tagging images	14
11.3	Publishing images	15
11.4	Docker Hub	15
11.5	Guia prático completo	15
12	Usando o build cache	16
12.1	Invalidação do cache	16
12.2	O problema do Dockerfile ingênuo	16
12.3	Dockerfile otimizado	17
12.4	Resultado da otimização	17
12.5	Princípio geral	17
13	Docker Compose	17
13.1	Dockerfile vs Compose file	17
13.2	Comandos básicos	18
14	Desenvolvendo com containers	18
14.1	Iniciando o ambiente de desenvolvimento com Docker Compose	18
14.2	Fazendo mudanças no serviço	19
14.3	Vantagens do ambiente containerizado	19

1 Introdução

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

2 Plataforma Docker

Docker é uma plataforma aberta para desenvolvimento, envio e execução de aplicações. Ele permite a separação da aplicação de sua infraestrutura, possibilitando o desenvolvimento de software de forma mais rápida. Com o Docker, é possível controlar a infraestrutura de uma aplicação da mesma maneira que se maneja a aplicação em si.

2.1 Containers

O Docker viabiliza a habilidade de empacotar e rodar uma aplicação em um ambiente isolado chamado *container*. O isolamento e a segurança proporcionados permitem rodar vários containers simultaneamente. Containers são leves e contêm tudo o que é necessário para rodar uma aplicação, eliminando a preocupação com o que está instalado no *host*. Além disso, é possível compartilhar containers durante o trabalho colaborativo.

O Docker provê ferramentas e uma plataforma para gerenciar os containers, seguindo o fluxo:

1. Desenvolver a aplicação e seus componentes de suporte usando containers.
2. O container se torna a unidade para distribuição e teste da aplicação.
3. Quando estiver pronto, implanta-se a aplicação em um ambiente de produção, seja como um container ou como um serviço.

2.2 Ciclo de desenvolvimento

O Docker permite o seguinte ciclo de trabalho: o desenvolvedor faz o código localmente e compartilha o trabalho com os colegas usando Docker containers. Os colegas usam o Docker para subir as aplicações em um ambiente de teste, rodando testes manuais e automáticos. Quando um desenvolvedor encontra *bugs*, ele pode corrigi-los no ambiente de desenvolvimento e reenviar as atualizações para o ambiente de teste e validação. Quando o teste é concluído, passar a aplicação corrigida para o cliente é simplesmente subir a imagem atualizada para o ambiente de produção.

A plataforma baseada em containers do Docker é excelente para a interoperabilidade, pois os containers podem ser rodados no ambiente local do desenvolvedor, em máquinas físicas, máquinas virtuais em um *data center*, na nuvem, entre outros ambientes.

2.3 Eficiência de recursos

O Docker é leve e rápido, provendo um bom custo-benefício quando comparado com máquinas virtuais baseadas em *hypervisor*. Docker se destaca quando se precisa fazer mais com menos recursos de hardware.

3 O que é o Docker? Containers vs Máquinas Virtuais

Imagine um cenário em que uma aplicação necessita de três componentes principais: um front-end React, uma API Python e um banco de dados PostgreSQL. Para trabalhar nesse projeto, seria necessário instalar Node, Python e PostgreSQL. Como garantir que as versões dessas ferramentas sejam as mesmas de todo o time? Ou do ambiente de CI/CD? Como evitar que outras ferramentas ou versões instaladas localmente interfiram no projeto?

Containers resolvem esses problemas. Um **container** é um processo isolado para cada componente da aplicação. Cada parte — o React frontend, a Python API e o banco de dados — roda em seu próprio ambiente isolado, completamente separado de tudo na máquina local.

3.1 Características dos Containers

- **Self-contained:** cada container possui tudo que precisa para funcionar, sem depender de nenhuma dependência local.
- **Isolado:** containers têm influência mínima da máquina local e de outros containers, aumentando a segurança da aplicação.
- **Independente:** deletar um container não afeta o funcionamento de outro.
- **Portável:** um container que roda em uma máquina vai rodar da mesma forma em outra, em um servidor ou na nuvem.

3.2 Containers vs Máquinas Virtuais

Uma **Máquina Virtual (VM)** é um sistema operacional completo, com seu próprio kernel, hardware drivers, programas e aplicações. Configurar uma VM apenas para rodar uma aplicação de forma isolada é pouco eficiente.

Um **container**, por outro lado, é um processo isolado com tudo necessário para a aplicação rodar. Se vários containers forem executados simultaneamente, eles compartilham o mesmo kernel do sistema operacional host, permitindo rodar mais aplicações com menos infraestrutura de hardware. Enquanto cada VM precisa de seu próprio sistema operacional completo, containers são mais leves e rápidos para iniciar.

4 Arquitetura Docker

O Docker usa uma arquitetura cliente-servidor. O *Docker client* comunica-se com o *Docker daemon*, que executa o trabalho de *build*, execução e distribuição dos Docker containers. O *Docker daemon* pode estar no mesmo sistema ou em um sistema remoto. A comunicação ocorre por meio de uma REST API, usando UNIX *sockets* ou uma interface de rede. Outro

cliente Docker é o Docker Compose, que permite trabalhar com aplicações que utilizam vários containers.

4.1 Docker Daemon

O Docker daemon (`dockerd`) escuta os *requests* da Docker API e gerencia os objetos Docker como imagens, containers, *networks* e volumes. Um daemon também pode se comunicar com outros daemons para gerenciar serviços do Docker.

4.2 Docker Client

O Docker client (`docker`) é a maneira primária de interagir com o Docker. Quando se utiliza comandos como `docker run`, o cliente envia os comandos para o `dockerd`, que então os processa. O comando `docker` utiliza a Docker API, e o cliente pode se comunicar com mais de um daemon.

4.3 Docker Desktop

Docker Desktop é uma aplicação para Mac, Windows ou Linux que permite buildar e compartilhar aplicações e microserviços em containers. Ele inclui o Docker daemon (`dockerd`), o Docker client (`docker`), Docker Compose, Docker Content Trust, Kubernetes e Credential Helper.

4.4 Docker Registries

Docker *registries* armazenam Docker images. O Docker Hub é um registro público que todos podem utilizar, e o Docker procura por imagens no Docker Hub por padrão. É possível também manter repositórios privados. Ao utilizar os comandos `docker pull` ou `docker run`, o Docker obtém imagens do registro configurado. Com o `docker push`, o Docker envia a imagem para o repositório configurado.

4.5 Docker Objects

Ao usar o Docker, trabalha-se com imagens, containers, *networks*, volumes, *plugins* e outros objetos.

4.5.1 Images

Uma *image* é um *template* com instruções para criar um Docker container. Muitas vezes, uma image é baseada em outra, com customizações adicionais. Por exemplo, é possível criar uma *build* baseada na imagem do Ubuntu que inclui o Apache Web Server e a aplicação desejada.

Para criar uma imagem própria, cria-se um **Dockerfile** com a sintaxe que define os passos necessários para gerar a image. Cada instrução do Dockerfile cria uma camada na image. Ao alterar o Dockerfile e refazer a *build*, somente as camadas modificadas são reconstruídas. Isso

é parte do que torna as imagens leves, pequenas e rápidas quando comparadas com serviços de virtualização.

4.5.2 Containers

Um container é uma instância em execução de uma imagem. É possível criar, iniciar, parar, mover ou deletar um container usando a Docker API ou CLI. Um container pode ser conectado a uma ou mais *networks*, receber armazenamento ou até servir de base para criar uma nova imagem a partir de seu estado atual.

Por padrão, um container é bem isolado de outros containers e da máquina *host*, mas é possível controlar o nível de isolamento da *network*, armazenamento ou outros subsistemas. Um container é definido por sua imagem e pelas opções de configuração fornecidas em sua criação. Quando um container é removido, qualquer mudança em seu estado que não foi armazenada de forma permanente desaparece.

4.6 Exemplo: comando docker run

O comando a seguir executa um container Ubuntu, conecta-se interativamente à sessão local de linha de comando e executa `/bin/bash`:

```
docker run -i -t ubuntu /bin/bash
```

Ao executar este comando (assumindo a configuração padrão de registro), ocorre a seguinte sequência:

1. Se a imagem `ubuntu` não existir localmente, o Docker a obtém do registro configurado, como se o comando `docker pull ubuntu` tivesse sido executado manualmente.
2. O Docker cria um novo container.
3. O Docker aloca um sistema de arquivos de leitura e escrita ao container, como sua camada final, permitindo que o container crie ou modifique arquivos e diretórios em seu sistema de arquivos local.
4. O Docker cria uma interface de rede para conectar o container à rede padrão, incluindo a atribuição de um endereço IP. Por padrão, containers podem se conectar a redes externas usando a conexão de rede da máquina *host*.
5. O Docker inicia o container e executa `/bin/bash`. Como o container roda interativamente e conectado ao terminal (devido às *flags* `-i` e `-t`), é possível fornecer entrada pelo teclado enquanto o Docker registra a saída no terminal.
6. Ao executar `exit` para encerrar o `/bin/bash`, o container para mas não é removido. É possível iniciá-lo novamente ou removê-lo.

5 Instalação

O Docker Desktop está disponível para Mac, Windows e Linux. No Linux, o Docker provê pacotes `.deb` e `.rpm` para as seguintes distribuições e arquiteturas: Ubuntu, Debian, Red Hat e Fedora. Existe uma versão experimental para distribuições baseadas em Arch, porém o Docker não testou nem verificou a instalação nessas distribuições.

5.1 Requisitos do sistema

Para executar o Docker Desktop no Linux, são necessários os seguintes requisitos:

- Kernel 64-bit e suporte a virtualização pela CPU
- Suporte a KVM para virtualização
- QEMU 5.2 ou superior (recomenda-se a versão mais recente)
- Sistema de inicialização `systemd`
- 4 GB de RAM

Vale ressaltar que o Docker Desktop no Linux roda uma máquina virtual.

5.2 Instalação no Ubuntu

Para instalar o Docker Desktop no Ubuntu, basta baixar o pacote mais recente e executar os seguintes comandos:

```
sudo apt-get update
sudo apt install ./docker-desktop-amd64.deb
```

6 Rodando o primeiro container e comandos de sobrevivência

Após a instalação, é possível rodar o primeiro container com o seguinte comando:

```
docker run -d -p 8080:80 docker/welcome-to-docker
```

O *frontend* dessa aplicação fica acessível na porta 8080, pelo endereço `http://localhost:8080` no navegador. Após a execução, o container aparece como rodando e a imagem passa a constar nas imagens locais.

6.1 Comandos básicos

6.1.1 Executando containers

```
docker run -d -p 8080:80 docker/welcome-to-docker
```

A flag `-d` executa o container em modo *detached* (em segundo plano). A flag `-p 8080:80` mapeia a porta 8080 do host para a porta 80 do container.

6.1.2 Listando containers

```
docker ps          # lista containers em execução
docker ps -a       # lista todos (incluindo os parados)
```

6.1.3 Parando containers

```
docker ps          # obter o ID do container
docker stop <ID>   # parar o container
```

Não é necessário digitar o ID completo — basta os caracteres suficientes para torná-lo único entre todos os containers existentes (inclusive os parados).

7 Imagens Docker

Uma **container image** (ou simplesmente imagem) é um pacote padronizado que contém todos os arquivos, binários, bibliotecas e configurações necessários para rodar um container. Por exemplo, uma imagem PostgreSQL contém os binários do banco de dados, arquivos de configuração e dependências; uma imagem de aplicação Python inclui o runtime Python, o código da aplicação e suas dependências.

7.1 Características das Imagens

- **Imutabilidade:** uma vez criada, a imagem não pode ser modificada. É possível apenas criar uma nova imagem ou adicionar camadas sobre uma existente.
- **Camadas (Layers):** imagens são compostas por camadas. Cada camada representa um conjunto de mudanças no sistema de arquivos — adição, remoção ou modificação de arquivos.

Essas duas características permitem estender imagens já existentes. Por exemplo, para construir uma aplicação Python, pode-se partir de uma imagem base do Python e adicionar camadas com as dependências e o código da aplicação.

7.2 Encontrando Imagens

O **Docker Hub** é o marketplace padrão para armazenar e distribuir imagens. O Docker categoriza as imagens em:

- **Docker Official Images:** repositórios curados pelo Docker, ponto de partida para a maioria dos usuários.
- **Docker Hardened Images:** imagens mínimas, seguras e prontas para produção, com praticamente zero CVEs.
- **Docker Verified Publishers:** imagens de alta qualidade publicadas por fornecedores verificados pelo Docker.

- **Docker-Sponsored Open Source:** imagens mantidas por projetos de código aberto patrocinados pelo Docker.

7.3 Comandos básicos para imagens

```
docker search docker/welcome-to-docker
docker pull docker/welcome-to-docker
docker image ls
docker image history docker/welcome-to-docker
```

O comando `docker pull` baixa a imagem do repositório, onde cada linha de download representa uma camada. O comando `docker image history` exibe as camadas que compõem a imagem.

8 Docker Registry

Um **registry** (registro) é um local centralizado para armazenar e compartilhar imagens de container. Pode ser público ou privado. O **Docker Hub** é o registro público padrão, acessível a todos. É possível também armazenar imagens em um registro local.

8.1 Outros registros

Além do Docker Hub, existem outros registros populares:

- Amazon Elastic Container Registry (ECR)
- Azure Container Registry (ACR)
- Google Container Registry (GCR)

8.2 Registry vs Repository

Apesar de relacionados, registro e repositório não são a mesma coisa. O **registro** é o local centralizado que armazena imagens, enquanto um **repositório** é uma coleção de imagens relacionadas dentro de um registro — como uma pasta que organiza imagens por projeto. Cada repositório contém uma ou mais imagens.

8.3 Fluxo prático: build, tag e push para o registro

```
git clone https://github.com/docker-samples/helloworld-demo-node
cd helloworld-demo-node
docker build -t SEU_USERNAME/docker-quickstart .
docker images
docker run -d -p 8080:8080 SEU_USERNAME/docker-quickstart
docker tag SEU_USERNAME/docker-quickstart SEU_USERNAME/docker-quickstart:1.0
docker push SEU_USERNAME/docker-quickstart:1.0
```

Após o push, a imagem estará disponível no Docker Hub, na seção *Tags* do repositório.

9 Entendendo as camadas de uma imagem

Uma imagem de container, quando criada, é formada por **camadas imutáveis**. Cada camada representa um conjunto de modificações no sistema de arquivos.

9.1 Exemplo de camadas

Considere uma imagem Python típica:

1. A primeira camada adiciona comandos básicos e um gerenciador de pacotes (**apt**).
2. A segunda camada adiciona o runtime Python e o **pip**.
3. A terceira camada copia o arquivo **requirements.txt** da aplicação.
4. A quarta camada instala as dependências listadas.
5. A quinta camada copia o código-fonte da aplicação.

Essa estrutura em camadas é benéfica porque permite a **reutilização** entre imagens. Se duas aplicações utilizam Python, ambas podem compartilhar as camadas que instalam o Python e o **pip**, tornando os builds mais rápidos e reduzindo o armazenamento necessário.

9.2 Empilhamento de camadas

1. Cada camada, ao ser instalada, é extraída em seu próprio diretório no sistema de arquivos local.
2. Quando um container é iniciado, um **sistema de arquivos unificado** é criado empilhando todas as camadas, gerando uma visão unificada.
3. O container então redireciona seu diretório raiz para essa visão unificada usando **chroot**.

9.3 Criando layers manualmente

Embora na prática se use Dockerfiles, é possível criar layers manualmente com **docker container commit** para entender o mecanismo interno.

9.3.1 Criando uma base image

```
# Iniciar container Ubuntu interativo
docker run --name=base-container -ti ubuntu
```

```
# Dentro do container: instalar Node.js
apt update && apt install -y nodejs
node -e 'console.log("Hello world!")'
```

```
# Em outro terminal: salvar como nova imagem
docker container commit -m "Add node" base-container node-base
```

```
# Verificar as layers
docker image history node-base
```

```
# Testar a nova imagem
docker run node-base node -e "console.log('Hello again')"
```

```
# Remover o container base
docker rm -f base-container
```

Uma **base image** é uma fundação para construir outras imagens. Qualquer imagem pode ser usada como base, mas algumas são criadas intencionalmente como blocos de construção.

9.3.2 Construindo uma app image

```
# Iniciar container a partir da node-base
docker run --name=app-container -ti node-base
```

```
# Dentro do container: criar um programa Node
echo 'console.log("Hello from an app")' > app.js
node app.js
```

```
# Em outro terminal: salvar como nova imagem com comando padrão
docker container commit -c "CMD node app.js" -m "Add app" \
    app-container sample-app
```

```
# Verificar as layers
docker image history sample-app
```

```
# Rodar a imagem final
docker run sample-app
```

```
# Limpar
docker rm -f app-container
```

A flag `-c` define configurações adicionais na imagem — neste caso, o comando padrão ao iniciar o container será `node app.js`.

Na prática, a maioria dos builds não usa `docker container commit`. Em vez disso, utiliza-se um **Dockerfile** que automatiza todos esses passos.

10 Anatomia do Dockerfile

O **Dockerfile** é um documento de texto que contém as instruções para criar uma imagem de container. Ele especifica comandos a executar, arquivos a copiar, comando de inicialização, entre outros.

10.1 Exemplo de Dockerfile

```
FROM python:3.13
WORKDIR /usr/local/app

COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

COPY src ./src
EXPOSE 8080

RUN useradd app
USER app

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

10.2 Principais instruções

- **FROM** *image*: especifica a imagem base utilizada no build.
- **WORKDIR** *path*: define o diretório de trabalho dentro da imagem, onde arquivos serão copiados e comandos serão executados.
- **COPY** *host-path image-path*: copia arquivos do host para dentro da imagem.
- **RUN** *command*: executa um comando durante o build (por exemplo, instalar dependências).
- **ENV** *name value*: define uma variável de ambiente disponível no container em execução.
- **EXPOSE** *port*: indica a porta que a imagem deseja expor.
- **USER** *user-or-uid*: define o usuário padrão para os comandos subsequentes.
- **CMD** ["command", "arg1"]: define o comando padrão executado ao iniciar um container a partir dessa imagem.

10.3 Sequência típica de um Dockerfile

Um Dockerfile geralmente segue esta ordem:

1. Determinar a imagem base (**FROM**).

2. Instalar as dependências da aplicação (RUN).
3. Copiar o código-fonte e binários relevantes (COPY).
4. Configurar a imagem final (EXPOSE, USER, CMD).

10.4 Exemplo prático: aplicação Node.js

Construindo um Dockerfile do zero para uma aplicação Node.js simples. O projeto base pode ser obtido clonando o repositório `docker/getting-started-todo-app` (branch `build-image-from-scratch`). O Dockerfile deve ser criado dentro da pasta `getting-started-todo-app/app/`.

É importante notar que o arquivo `Dockerfile` não tem extensão. Alguns editores podem tentar adicionar uma automaticamente.

```
FROM node:22-alpine
WORKDIR /app
COPY . .
RUN yarn install --production
CMD ["node", "./src/index.js"]
```

Cada instrução corresponde a um passo: a imagem base é o Node.js 22 com Alpine Linux (leve e enxuta); o diretório de trabalho é `/app`; todos os arquivos do projeto são copiados; as dependências de produção são instaladas via Yarn; e o comando padrão inicia a aplicação.

Este Dockerfile não segue todas as boas práticas (cache de build, usuário não-root, multi-stage builds). O comando `docker init` pode ser usado para gerar automaticamente um Dockerfile otimizado.

11 Construindo, tagueando e publicando imagens

11.1 Building images

O comando mais básico para construir uma imagem é:

```
docker build .
```

O `.` indica o *build context* — o diretório onde o builder encontrará o Dockerfile e os arquivos referenciados. Ao executar, o builder baixa a imagem base (se necessário) e executa as instruções do Dockerfile. Sem uma tag, a imagem recebe apenas um ID (hash SHA256), que não é fácil de lembrar.

11.2 Tagging images

Tags dão nomes memoráveis às imagens. A estrutura completa de um nome de imagem é:

```
[HOST[:PORT_NUMBER]/]PATH[:TAG]
```

- **HOST**: registro onde a imagem está. Padrão: `docker.io`.
- **PORT_NUMBER**: porta do registro (se um host foi especificado).
- **PATH**: caminho no formato `[NAMESPACE/]REPOSITORY`. Se nenhum namespace é dado, usa-se `library`.
- **TAG**: identificador de versão. Padrão: `latest`.

Para dar tag durante o build, usa-se a flag `-t`:

```
docker build -t meu-usuario/minha-imagem .
```

Para adicionar outra tag a uma imagem já existente:

```
docker image tag meu-usuario/minha-imagem outro-usuario/outra-imagem:v1
```

11.3 Publishing images

Para enviar a imagem ao registro, é necessário estar autenticado e usar o comando `docker push`:

```
docker login
docker push meu-usuario/minha-imagem
```

11.4 Docker Hub

Para compartilhar Docker images, é necessário um local para armazená-las. O Docker Hub é o registro padrão, provendo tanto um local para armazenar images próprias quanto para encontrar images de outros usuários.

Ao escolher images, o Docker Hub fornece categorias de images confiáveis:

- **Docker Official Images (DOI)**: images selecionadas dos softwares mais populares, seguindo boas práticas e atualizadas regularmente.
- **Docker Hardened Images (DHI)**: images desenhadas para reduzir a superfície de ataque — seguras, simples e praticamente sem vulnerabilidades.

11.5 Guia prático completo

```
# Clonar o repositório
git clone https://github.com/docker/getting-started-todo-app
cd getting-started-todo-app
```

```
# Construir a imagem
docker build -t SEU_USERNAME/concepts-build-image-demo .
```

```
# Verificar a imagem criada
docker image ls
```



```
# Visualizar histórico de layers
docker image history SEU_USERNAME/concepts-build-image-demo
```

```
# Enviar ao Docker Hub
docker push SEU_USERNAME/concepts-build-image-demo
```

O comando `docker image history` mostra as camadas da imagem, distinguindo as que foram adicionadas pelo usuário das herdadas da imagem base. Se ao fazer push receber o erro `requested access to the resource is denied`, verifique se está logado (`docker login`) e se o username na tag está correto.

12 Usando o build cache

Ao executar `docker build`, o Docker executa cada instrução do Dockerfile criando uma camada para cada comando, na ordem especificada. Para cada instrução, o Docker verifica se pode reutilizar o resultado de um build anterior. Se encontrar uma correspondência, ele usa o resultado em cache ao invés de re-executar a instrução, tornando o build mais rápido e eficiente.

12.1 Invalidação do cache

O cache é invalidado nas seguintes situações:

- Qualquer mudança no comando de uma instrução `RUN` invalida aquela camada.
- Qualquer alteração nos arquivos copiados com `COPY` ou `ADD` — seja no conteúdo ou em propriedades como permissões.
- Uma vez que uma camada é invalidada, **todas as camadas subsequentes** também são invalidadas, prevenindo inconsistências.

12.2 O problema do Dockerfile ingênuo

Considere o Dockerfile inicial:

```
FROM node:22-alpine
WORKDIR /app
COPY . .
RUN yarn install --production
EXPOSE 3000
CMD ["node", "./src/index.js"]
```

O primeiro build leva cerca de 20 segundos. Um rebuild sem alterações leva aproximadamente 1 segundo, pois todas as camadas estão em cache. Porém, qualquer alteração no código-fonte invalida o `COPY . .`, forçando o `RUN yarn install` a executar novamente — mesmo que as dependências não tenham mudado.

12.3 Dockerfile otimizado

A solução é copiar primeiro apenas os arquivos de dependência, instalar, e depois copiar o restante:

```
FROM node:22-alpine
WORKDIR /app
COPY package.json yarn.lock ./
RUN yarn install --production
COPY . .
EXPOSE 3000
CMD ["node", "src/index.js"]
```

Também deve-se criar um arquivo `.dockerignore` com o conteúdo `node_modules` para evitar copiar dependências locais.

12.4 Resultado da otimização

Com essa estrutura, ao alterar apenas o código-fonte (por exemplo, `src/static/index.html`), as camadas de `COPY package.json` e `RUN yarn install` permanecem em cache. O rebuild cai de 16 segundos para aproximadamente 1,2 segundo.

12.5 Princípio geral

Instruções que mudam com **menor frequência** devem ficar no topo do Dockerfile (como instalação de dependências), enquanto instruções que mudam **com mais frequência** devem ficar por último (como a cópia do código-fonte). Dessa forma, o cache é aproveitado ao máximo.

13 Docker Compose

Quando uma aplicação envolve múltiplos containers — bancos de dados, caches, filas de mensagens, entre outros serviços — surge a questão de como orquestrá-los. Uma boa prática é que cada container faça uma única função e a faça bem.

Embora seja possível usar vários comandos `docker run` para iniciar múltiplos containers, isso exige lidar manualmente com networks e diversas flags de configuração. O **Docker Compose** resolve esse problema ao permitir definir todos os containers e suas configurações em um único arquivo YAML. Qualquer pessoa que clone o repositório pode subir toda a aplicação com um único comando.

13.1 Dockerfile vs Compose file

O **Dockerfile** contém instruções para construir uma *container image*, enquanto o **Compose file** define os containers que serão executados e suas configurações. Frequentemente,

o arquivo Compose referencia um Dockerfile para buildar a imagem usada em um serviço específico.

13.2 Comandos básicos

```
# Clonar o projeto de exemplo
git clone https://github.com/dockersamples/todo-list-app
cd todo-list-app
```

```
# Subir todos os serviços
docker compose up -d --build
```

```
# Derrubar todos os serviços
docker compose down
```

O arquivo `compose.yaml` dentro do projeto define todos os serviços, suas imagens, portas, volumes, networks e demais configurações necessárias.

14 Desenvolvendo com containers

14.1 Iniciando o ambiente de desenvolvimento com Docker Compose

Para iniciar um projeto existente com Docker, basta clonar o repositório e utilizar o Docker Compose:

```
git clone https://github.com/docker/getting-started-todo-app
docker compose watch
```

Ao inicializar a aplicação, vários containers são criados, cada um servindo uma função específica:

- **React frontend:** um container Node rodando o servidor de desenvolvimento React, usando Vite.
- **Node backend:** o backend provê uma API que permite retirar, criar e deletar itens de tarefas.
- **MySQL database:** o banco de dados para armazenar a lista de itens.
- **phpMyAdmin:** uma interface web para interagir com o banco de dados, acessível em `http://db.localhost`.
- **Traefik proxy:** uma aplicação proxy que direciona *requests* para o serviço correto. Ela redireciona requisições para `localhost/api/*` ao backend, requisições para `localhost/*` ao frontend, e requisições para `db.localhost` ao phpMyAdmin. Isso permite acessar todas as aplicações pela porta 80, ao invés de usar portas diferentes para cada serviço.

14.2 Fazendo mudanças no serviço

É possível alterar diversos aspectos da aplicação diretamente nos arquivos do projeto:

- **Greeting:** populado pela chamada à API em `/api/greeting`, alterável em `backend/src/routes/get`
- **Placeholder text:** alterável no atributo *placeholder* do componente `Form.Control` em `client/src/components/AddNewItemForm.jsx`.
- **Cor de fundo:** alterável pelo valor da chave `background-color` em `client/src/index.css`.

14.3 Vantagens do ambiente containerizado

Foi possível iniciar um ambiente de desenvolvimento completo sem nenhuma instalação ou configuração prévia de dependências. O ambiente de containers proveu tudo o que era necessário para o desenvolvimento, sem a necessidade de instalar Node, MySQL ou qualquer outra dependência na máquina local — apenas o Docker e um editor de código.

Além disso, as mudanças realizadas nos arquivos locais são automaticamente sincronizadas com os containers, pois os arquivos do diretório de projeto local são compartilhados com o ambiente do container. Os processos em execução monitoram e respondem a alterações nos arquivos em tempo real.

Referências